Documentation API

OpenAPI Specification

PLAN DU COURS

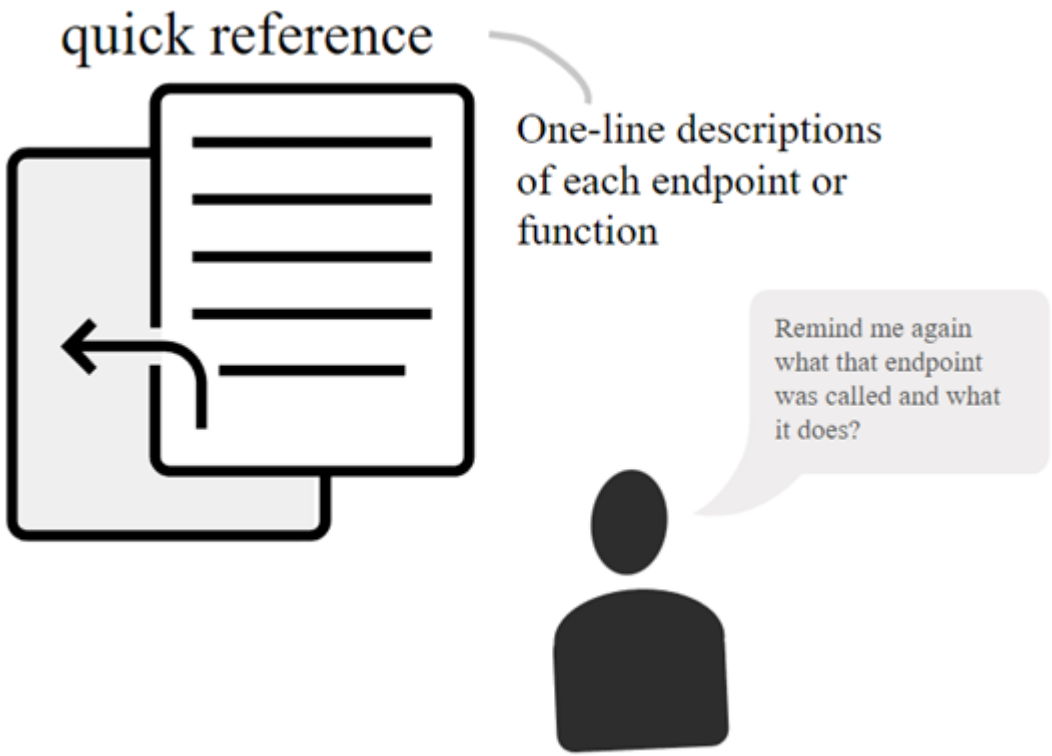
- Introduction
- OpenAPI Spécification
- Implémentations
- Intégration et Configuration de **Spring Doc**

- TP : Exposition de tous les Web Services : Chambre (déjà fait), Bloc, Etudiant, Foyer, Reservation, Universite
- TP : Test de tous les Web Services avec Postman et avec Swagger (Spring Doc

Introduction

Que signifie la documentation API ?

C'est un manuel de référence précis qui contient les informations nécessaires pour travailler avec l'API, notamment des détails sur les fonctions, les classes, les types de retour et les arguments.

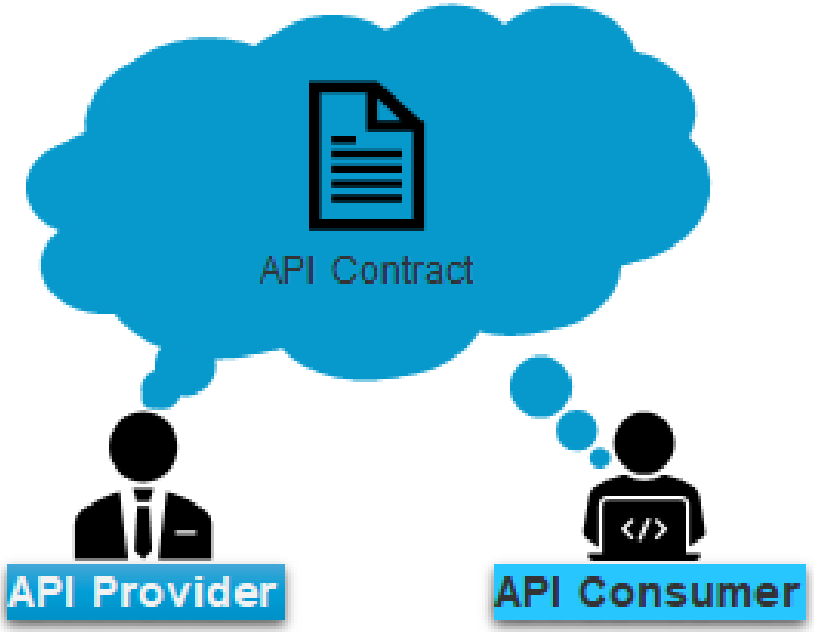


Introduction

Pourquoi ?

Les API ont pour vocation de servir à de nombreux développeurs.
Par conséquent, le développement d'une API nécessite une documentation accessible et facilement utilisable.

Cette dernière doit toujours être à jour au fur et à mesure que le code et les fonctionnalités de l'API évoluent.



OpenAPI Specification

La documentation d'un service RESTful consiste essentiellement à décrire les détails des requêtes HTTP qu'elle consomme et des réponses HTTP qu'elle produit.

Préparer une documentation de qualité est une tâche difficile.

Il est donc important d'utiliser des outils appropriés à cette tâche.

Les formats de description d'API tels que **la spécification OpenAPI** (Swagger v3) ont permis de simplifier la création et la maintenance de la documentation.

Elle garantit également que votre documentation soit à jour au fur et à mesure de l'évolution de votre API.

Implémentations

Pour l'intégration d'OAS dans notre application, nous devons faire appel à l'implémentation qui nous convient.

Language	Implémentation
Java	springdoc-openapi
.Net	Microsoft.OpenApi.net
Node.js	oas3-remote-refs

Intégration SpringDoc

Pour commencer, nous allons simplement ajouter la dépendance **springdoc-openapi-starter-webmvc-ui** dans le fichier **pom.xml.** Et puis faire **maven update** de votre projet.

```
<dependency>
   <groupId>org.springdoc</groupId>
   <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
   <version>2.1.0</version>
</dependency>
```

C'est tout, aucune configuration supplémentaire n'est nécessaire.
La documentation sera disponible au format HTML en utilisant l'outil swagger-ui.

Intégration SpringDoc

Démarrer votre application Spring Boot :
Tomcat started on port(s): 8089 (http) with context path '/tpfoyer'

La page Swagger UI (Spring Doc) sera alors disponible à l'adresse:
http://**server:port/context-path/swagger-ui/index.html**

serveur : Le nom ou l'IP du serveur : **localhost**
port : Le port du serveur : **8089**
context-path : Le chemin du contexte de l'application : **tpfoyer :**

http://localhost:8089/tpfoyer/swagger-ui/index.html

Intégration SpringDoc

Configuration SpringDoc

Nous pouvons aussi personnaliser la documentation grâce aux annotations:

- **@Tag** (@Api in swagger 2): permet d'ajouter une description pour chaque classe RestController.
- **@Operation**(@ApiOperation in swagger 2): permet d'ajouter une description pour chaque classe RestController.

```
@Tag(name = "Gestion Chambre")
@RestController
@AllArgsConstructor
@RequestMapping("/chambre")
public class ChambreRestController {

IChambreService chambreService;

// http://localhost:8089/tpfoyer/chambre/retrieve-all-chambres
@Operation(description = "récupérer toutes les chambres de la base de données")
@GetMapping("/retrieve-all-chambres")
public List<Chambre> getChambres() {
        return chambreService.retrieveAllChambres();}
```

File Edit View Navigate Code Refactor Build Run Tools VCS Window Help tp-foyer - ChambreRestController.java

tp-foyer > src > main > java > tn > esprit > tpfoyer > control > ChambreRestController

TpFoyerApplication

Project

- tp-foyer C:\Work\workspace-intellij\tp-foyer
  - .idea
  - .mvn
  - src
    - main
      - java
        - tn.esprit.tpfoyer
          - control
            - BlocRestController
            - ChambreRestController
            - EtudiantRestController
            - FoyerRestController
            - ReservationRestController
            - UniversiteRestController
          - entity
          - repository
          - service
          - TpFoyerApplication
      - resources
    - test
  - target
  - .gitignore
  - HELP.md

ChambreRestController.java | BlocServiceImpl.java | FoyerServiceImpl.java | BlocRestController.java | Etud

```
import org.springframework.web.bind.annotation.*;
import tn.esprit.tpfoyer.entity.Chambre;
import tn.esprit.tpfoyer.service.IChambreService;
import java.util.List;

@Tag(name = "Gestion Chambre")
@RestController
@AllArgsConstructor
@RequestMapping("/chambre")
public class ChambreRestController {

    5 usages
    IChambreService chambreService;

    // http://localhost:8089/tpfoyer/chambre/retrieve-all-chambres
    @Operation(description = "récupérer toutes les chambres de la base de données")
    @GetMapping("/retrieve-all-chambres")
    public List<Chambre> getChambres() {
        List<Chambre> listChambres = chambreService.retrieveAllChambres();
        return listChambres;
    }
    // http://localhost:8089/tpfoyer/chambre/retrieve-chambre/8
    @GetMapping("/retrieve-chambre/{chambre-id}")
    public Chambre retrieveChambre(@PathVariable("chambre-id") Long chId) {
        Chambre chambre = chambreService.retrieveChambre(chId);
        return chambre;
    }
```

Run: TpFoyerApplication

Console | Actuator

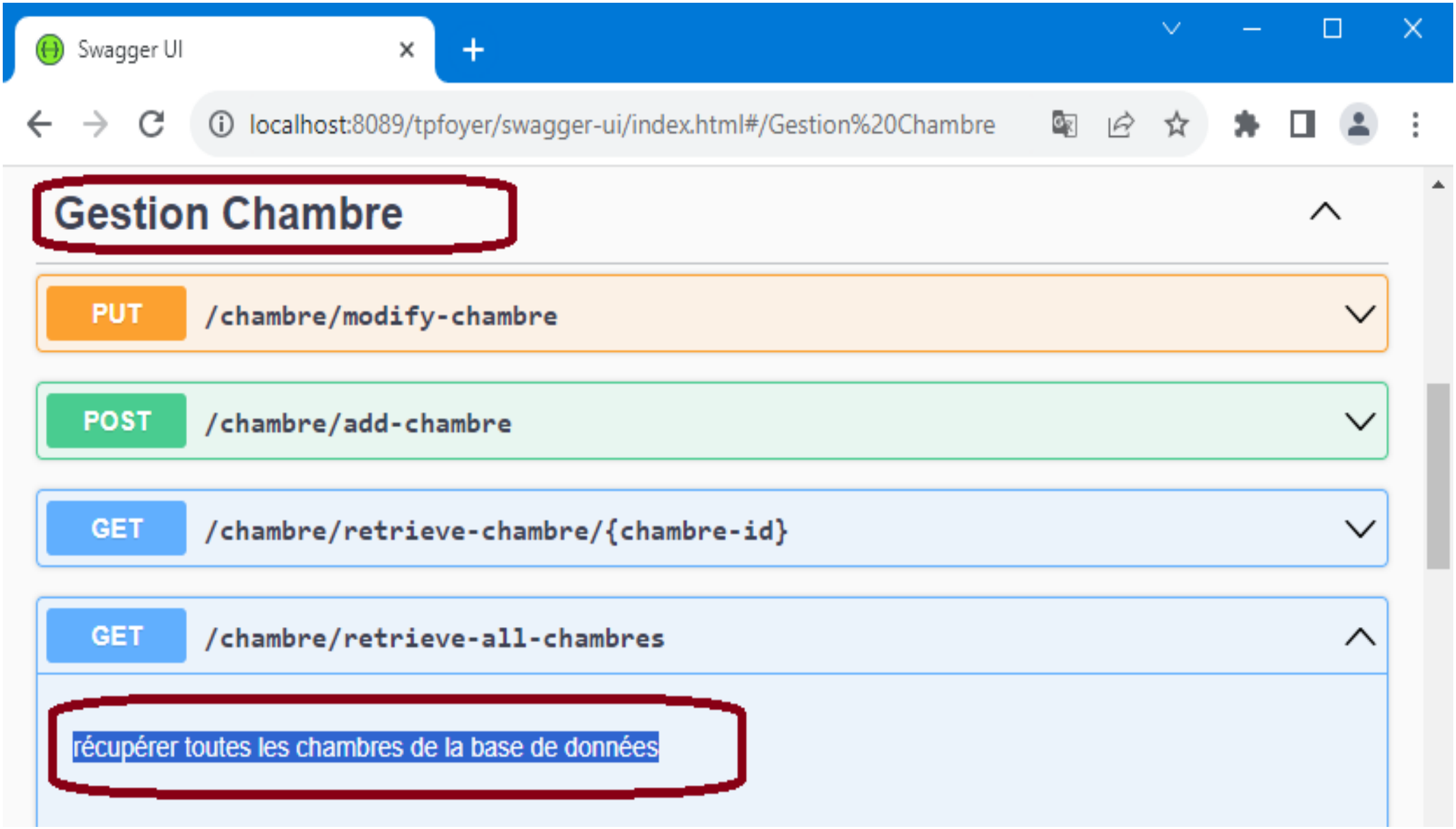
```
figuration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed do
mbedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8089 (http) with context path '/tpfoyer'
.tpfoyer.TpFoyerApplication    : Started TpFoyerApplication in 6.493 seconds (process running for 8.463)
.[.[localhost].[/tpfoyer]      : Initializing Spring DispatcherServlet 'dispatcherServlet'
```

Version Control | Run | TODO | Problems | Spring | Terminal | Endpoints | Services | Profiler | Build | Database Changes | Dependencies | SonarLin

TpFoyerApplication: Failed to retrieve application beans snapshot: // :application=* (10 minutes ago)

16:1 CRLF UTF-8 4 spaces

Configuration SpringDoc



TP

Implémenter les Web Service CRUD pour toutes les entités (Service + Controller) : **Bloc**, **Foyer**, **Reservation**, **Universite**, **Etudiant**, (**Chambre** déjà implémentée la semaine dernière normalement).

Utiliser **Swagger (Spring Doc) et Postman** pour tester les services implémentés.